Concept Drift Detection Using Optimal Transport

Afnane Touhar[1], Meriem Afaf Haddou[1], Leila Hamdad[1, 2], Karima Bentachba[1, 3], and Souhila Sadeg[1, 3]

[1]Ecole Nationale Superieure d'Informatique ESI (ex.INI), BP 68M - 16270 Oued Smar, Alger, Algeria
[2]Laboratoire de la Communication dans les Systèmes Informatiques (LCSI), BP 68M - 16270 Oued Smar, Alger, Algeria
[3]Laboratoire des Methodes de Conception de Systèmes (LMCS), BP 68M - 16270 Oued Smar, Alger, Algeria

Abstract

In machine learning, data streams present challenges due to their large size, speed, and dynamic nature. As data distributions shift over time — a phenomenon called concept drift — model performance deteriorates, leading to reduced accuracy and increased false predictions. This requires frequent retraining to maintain effectiveness.
To address this, drift detection systems are crucial for timely model updates. While monitoring a model's performance in production is a straightforward method, it often results in late detection, as the model's effectiveness has already declined before the drift is identified. In this work, we tackle the challenge of concept drift detection using a proactive approach based on optimal transport (OT). OT is a mathematical framework that efficiently maps one probability distribution to another while minimizing transformation costs. By leveraging OT to compare data distributions over time, our method enables the rapid and accurate detection of changes, anticipating model performance degradation before it becomes critical. Unlike reactive methods that detect drift only after performance declines, our approach monitors the underlying data distributions directly. This allows earlier intervention and supports multidimensional data streams, making it well-suited for dynamic and non-stationary environments.
This dissertation presents a drift detection system grounded in OT metrics, capable of both identifying drift and determining its type. The system has been tested extensively on synthetic and real-world datasets. The results demonstrate its accuracy, robustness, and responsiveness, offering a reliable solution to maintaining model performance in evolving data contexts.

1 Introduction

Governments and corporations are increasingly generating vast amounts of streaming data, driving the need for sophisticated data analytics and machine learning techniques to enhance predictive modeling and decision-making processes. However, the evolving nature of new products, markets, and customer behaviors poses a significant challenge: concept drift (CD), which poses a substantial obstacle to maintaining model accuracy over time. Concept drift means that the statistical properties of the target variable, which the model is trying to predict, change over time in unforeseen ways. If the concept drift occurs, the induced pattern of past data may not be relevant to the new data, leading to poor predictions and decision outcomes [1]. Rapid detection and effective adaptation to this phenomenon are therefore critical to maintain the

efficiency of models in streaming environments. One simple and easy-to-implement approach is to monitor the real-time performance of machine learning models. A drop in performance may indicate that a distributional change has occurred. Nevertheless, this approach has a major drawback: late detection. Waiting for the model's performance to decline before detecting the drift can lead to serious negative consequences, especially in critical contexts like healthcare and public safety. Therefore, it is essential to consider a proactive approach that allows for rapid detection of these changes before the model becomes obsolete. To achieve this, the evolution of underlying data distributions must be continuously monitored, using robust and reliable metrics that account for the structures of these distributions over time, and detect their changes. Optimal transport emerges as a very promising tool. Existing drift detection methods are often good at determining when drift has occurred. However, few retrieve information about how the drift came to be present in the stream. Hence, determining the impact of the type of drift on adaptation is difficult. Filling this gap, we designed a OT based solution to detect the CD and his type once it occurs and adapt the model to the new data.

OT theory, originally developed by Gaspard Monge [2] and later expanded by Leonid Kantorovich, has had a profound impact on mathematics, physics, and computer science. In recent years, OT has gained significant attention in the field of machine learning (ML), where it contributes in two major ways: (i) as a loss function and (ii) for handling probability distributions. OT introduces a metric between distributions, commonly referred to as the Wasserstein distance, also known as Kantorovich metric, or the Earth Mover Distance (EMD). This metric is part of the Integral Probability Metrics (IPMs) family, and in many ML tasks, such as generative modeling, it is often preferred over measures like Kullback-Leibler (KL) divergence due to its advantageous topological and statistical properties [3]. It is also used to solve problems related to domain adaptation [4] [5] [6], community detection [7], neural network architecture search [8] [9], and more. This tool is efficient in capturing the complex relationships between data distributions, studying similarities between point clouds, and detecting changes and anomalies, including concept drift.

The rest of the paper is organized as follows: Section 2 reminds the main principles of CD and OT. Section 3 reviews related work and recent developments in CD detection using OT. Section 4 is dedicated to our theoretical contribution which leads to a comprehensive drift detection system grounded in OT metrics. This system is not only capable of detecting the occurrence of drift but also provides insights into the type of drift, whether it involves changes in data distributions, labels, or other structural properties. An extensive experimental study is presented in Section 5 before our conclusion in Section 6.

2 Background

In this section, we first give the formal definition, the sources and the commonly defined types of concept drift. Then we give an overview of OT.

2.1 Concept Drift

In supervised machine learning tasks, each data instance is defined by a pair of feature or covariate vectors X, and a target or response variable y. In [10], the authors introduced a probabilistic definition to describe a time-varying concept as the joint distribution of X and y at time t, denoted $P_t(X, y)$.
Concept drift is generally framed within the context of stream learning, as a data stream is defined as a continuous, potentially unbounded sequence of data elements. This contrasts with dataset changes in a batch learning scenario, where the data is entirely stored in memory and processed all at once. Changes are characterized between the learning and testing probability distributions.
Formally, concept drift is defined as follows: Given a time period $[0, t]$, a sample set denoted $S_{0,t} = d_0, ..., d_t$ where $d_i = (X_i, y_i)$ is

a data observation or instance, X_i is the feature vector, and y_i represents the corresponding label, $S_{0,t}$ follows a certain distribution $F_{0,t}(X, y)$. Concept drift occurs at time $t+1$ if $F_{0,t}(X, y) \neq F_{t+1,+\infty}(X, y)$, as denoted by [10]: $\exists t : P_t(X, y) \neq P_{t+1}(X, y)$.
Concept drift is also defined as a change in the joint distribution between two time instances t and $t+w$ [11], where t can be a specific point or time interval, and w refers to the time window in which the distribution change is verified. Consequently, concept drift occurs when : $P_t(X, y) \neq P_{t+w}(X, y)$

2.1.1 Concept Drift Sources

The concept representing the joint probability is defined by [1] as: $P_t(X, y) = P_t(X)P_t(y|X)$ Thus, the change in joint probability can be caused by one of the following sources:

- **Source 1 :** $P_t(X) \neq P_{t+1}(X)$ and $P_t(y|X) = P_{t+1}(y|X)$, which implies that the change occurs in the distribution $P(X)$ while $P(y|X)$ remains unchanged. This means that the input data changes, but the way decisions are made based on this data does not change. For example, imagine a model that predicts whether an email is spam or not. If the frequency of words changes (e.g., new words appear more frequently), but the emails containing these words do not change their labels (spam or not spam), then this is a virtual drift. The figure 1 illustrates this source in a two-dimensional feature space. We can see that the distribution $P(X)$ at time $t+1$ is different from that at time t. New elements, shown in red, have been added. This type of drift does not affect the decision boundary, shown in blue. It is called virtual drift.

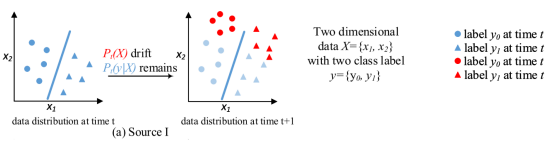

Figure 1: Source 1 of Concept drift [12]

- **Source 2 :** $P_t(y|X) \neq P_{t+1}(y|X)$ but $P_t(X) = P_{t+1}(X)$. This drift, also known as real drift, cause a change in the decision boundary and a decrease in learning accuracy in supervised scenarios. This means that even if the input data remains constant, the way we need to make decisions based on this data changes. For example, in the email prediction model, if the criteria for classifying an email as spam or not spam change (e.g., some words that were not previously indicators of spam become indicators), then this is real drift. However, this does not apply to the unsupervised case, as class labels are not available. The figure 2 illustrating this concept shows that although the data points remain in the same positions, the decision boundary (blue line) has changed.

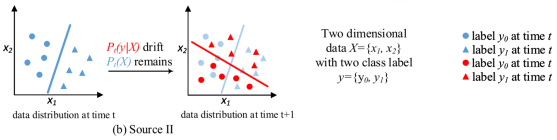

Figure 2: Source 2 of Concept drift [12]

- **Source 3 :** A combination of the two previous sources, where $P_t(X) \neq P_{t+1}(X)$ and $P_t(y|X) \neq P_{t+1}(y|X)$. Concept drift focuses on the change in both $P(y|X)$ and $P(X)$, as these two changes carry important information about the learning environment. This means that both the input data changes and the way decisions are made based on this data also changes. For example, suppose not only the words in the emails change (as in virtual drift), but also the criteria for determining whether an email is spam or not changes (as in real drift). The figure 3 illustrates this source. We observe that the distribution has changed with the addition of new elements shown in red, as well as the decision boundary, represented by the red line.

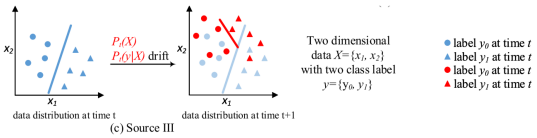

Figure 3: Source 3 of Concept drift [12]

2.1.2 Concept Drift Types

Concept drift occurs when the statistical properties of the target variable, which the model aims to predict, change unexpectedly over time. This variation can happen at different speeds and follow different patterns, such as sudden or gradual changes. Assuming that the concept of a data instance sequence shifts from S_1 to S_2, four types of drift are identified: sudden, gradual, incremental, and recurrent [12]. The figures presented in this section provide an illustration of these main types, assuming one-dimensional data, where a source is characterized by the mean of the data.

- **Sudden Concept Drift :** occurs when the model built on a certain concept S_1 becomes obsolete because the data distribution changes abruptly, as shown in Figure 4 . All new data points now belong to a different concept S_2. This sudden change in the data distribution affects the relationship between the model's inputs and outputs, making predictions less accurate.
- **Gradual concept drift :** refers to a progressive change where two sources, S_1 and S_2, are active simultaneously for a period of time. Over time, the probability of receiving an instance from source S_1 decreases, while the probability of receiving an instance from source S_2 increases, until S_2 completely replaces S_1, as illustrated in Figure. This scenario represents a gradual shift in the data distribution, where an initial concept is slowly replaced by a new one.
- **Incremental concept drift :**describes a particular case of gradual drift, where the change in the data distribution occurs subtly, with gradual variations in the probabilities of data instances arriving. Over time, source S_1 is progressively replaced by source S_2, as shown in Figure.
- **Recurrent concept drift:** refers to data changes that reappear after some time, without necessarily following a periodic pattern, as shown in Figure. This means that machine learning models may encounter variations in the data that occur at irregular intervals, making it difficult to predict exactly when these changes will happen.

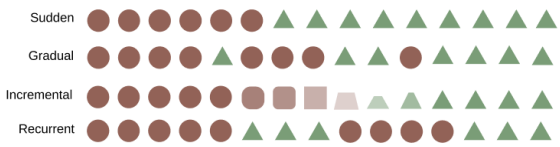

Figure 4: Concept drift types.

2.2 Optimal Transport (OT)

OT is an old field of research that emerged in the 18th century [2]. It sits at the intersection of probability, partial differential equations, geometry, and optimization. Over time, OT has developed multiple formulations, reflecting both the theoretical advancements and practical applications of this field. In this section, we present how this domain has evolved by highlighting its various formulations.

2.2.1 Monge Formulation

OT is a mathematical method designed to transport resources while minimizing transportation costs. It was introduced in 1781 by the renowned mathematician Gaspard Monge in his work "Théorie des déblais et des remblais". The problem was initially framed as moving a pile of sand (debris) to another pile (embankment), which can be represented by two probability measures μ (for the debris) and ν (for the embankment). The goal is to find a transport map T such that the sand at position x is fully moved to $T(x)$, and the volume of the embankment matches that

of the debris. This relationship is expressed as: $\nu(B) = \mu(T^{-1}(B))$.
This means ν is the image of μ through T. The problem is formalized in the framework of Polish spaces (separable and complete metric spaces). Monge's problem is to minimize the total transportation cost, defined as: $\int_X c(x, T(x))\, d\mu(x)$
Here, $c(x, y)$ represents the unit transport cost from x to y. Monge's goal is to minimize this cost over all possible transport maps T such that ν is the image of μ under T. This is summarized by solving the following equation: $\inf_{T:T_*\mu=\nu} \int_X c(x, T(x))\, d\mu(x)$

2.2.2 Monge-Kantorovitch Formulation

One of the major advances in OT was made by mathematician Leonid Kantorovich, the founder of linear programming. He addressed the limitations of Monge's original formulation by proposing a second formulation called the Monge-Kantorovich problem, which can be solved using linear programming [9]. It is defined as follows: Let X and Y be two Polish spaces, and let $\mu \in P(X)$ and $\nu \in P(Y)$, where Π denotes the set of probability measures on $X \times Y$ that have μ and ν as marginals. These probabilities are called transport plans that send μ to ν. Additionally, a transport cost function c: $X \times Y \to R^+$ is defined. Kantorovich's problem seeks to find a $\pi \in \Pi(\mu, \nu)$ that minimizes the cost:

$$\inf_{\pi \in \Pi(\mu,\nu)} \int_{X \times Y} c\, d\pi := \inf_{\pi \in \Pi(\mu,\nu)} I[\pi] \tag{1}$$

Such a probability π is called the OT plan, and the value of the minimum is referred to as the OT cost [9].
Unlike Monge's formulation, Kantorovich's approach avoids the issue of finding a transport map, since $\mu \otimes \nu \in \Pi(\mu, \nu)$, where the tensor product $\mu \otimes \nu$ combines two probability measures on distinct spaces to form a new probability measure on the product space. This ensures that $\Pi(\mu, \nu)$ is non-empty, allowing for transport plans that distribute mass from a source point to several target points.
Additionally, OT plans minimize the distance between two measures, known as the Wasserstein-p distance, where p denotes the order of the distance, and d is a metric [9]:

$$W_d^p(\mu, \nu) = \inf_{\pi \in \Pi(\mu,\nu)} \left(\int_{X \times Y} d(x, y)^p\, d\pi(x, y) \right)^{1/p} . \tag{2}$$

In this formulation, $\Pi(\mu, \nu)$ represents the set of OT plans between distributions μ and ν, and $d(x, y)$ is a distance metric defined on the space $X \times Y$. Once transformed into a linear programming problem, this formulation can be efficiently solved using the Simplex algorithm [9].

2.2.3 Regularized Wasserstein Distance (Sinkhorn Distance)

The regularized Wasserstein distance, also known as the Sinkhorn distance, is a smoothed version of the classical Wasserstein distance that incorporates an entropic regularization term. This modification results in a more efficient computation of the OT problem [13]. The Sinkhorn distance can be computed using the Sinkhorn matrix scaling algorithm, which is significantly faster than traditional transport solvers [14]. The optimization problem between distributions a and b is formulated as follows: $\gamma = \langle \gamma, M \rangle_F + \text{reg} \cdot \Omega(\gamma)$ where :

- M is the cost matrix between distributions a and b,
- $\langle \gamma, M \rangle_F$ is the Frobenius inner product of matrices γ and M
- $\Omega(\gamma)$ is the entropic regularization term, given by: $\Omega(\gamma) = \sum_{i,j} \gamma_{ij} \log(\gamma_{ij})$.
- reg is the regularization parameter, with $reg > 0$.

This regularization accelerates the computation while still yielding an approximation of the OT plan.

3 Related work

In the literature, class labels are used in the supervised case to monitor distributional changes, such as monitoring predictive errors to detect shifts. However, in the unsupervised case,

methods consist of comparing data distributions across two different time intervals to detect distributional changes. Thus, the distribution comparison approach can be applied in both unsupervised and supervised cases.

Zhao and Koh in [15] developed a Wasserstein distance based solution capable of detecting both sudden and gradual concept drift. The proposed solution aims to detect sudden changes in data distribution, as well as slow and gradual shifts, by measuring distributional changes and interpreting their significance. They used sliding windows to store data distributions, the reference window represents the old data distribution, while the current window represents the new data distribution, as shown in figure 5 , and the Wasserstein distance to measure the minimal cost required to transform one distribution into another. By comparing the data distributions between the reference window and the current window, the Wasserstein distance quantifies the amount of change between the two distributions. When the measured distance exceeds a predefined threshold, a concept drift is flagged. This method is particularly effective because it accounts for the geometric structure of the data, providing a robust and sensitive measure of distributional changes. Sudden concept drift is detected when the distributional change between two points in time exceeds a threshold within a short time frame. Gradual concept drift is characterized by slow changes over a longer period, it is identified by continuously monitoring distributional changes over time, ensuring the change progresses gradually.

Once a concept drift is detected, Zhao and Koh [15] evaluated feature importance using methods such as SHapley Additive Explanations (SHAP), Local Interpretable Model-agnostic Explanations (LIME), and Permutation Importance (PI). These methods help to understand which features contribute to the concept drift.

Experimental results demonstrated the effectiveness of this solution in detecting both sudden and gradual concept drift. The use of Wasserstein distance allows for scalable, label-independent drift detection. The framework's ability to identify and describe changes in feature importance ensures that models can adapt to new data distributions, thereby maintaining their predictive performance over time.

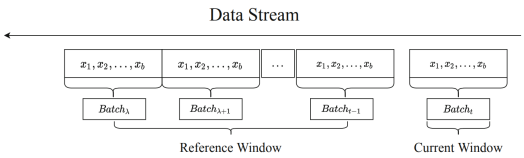

Figure 5: Batch and sliding window [15].

Laha and Verma in [16] classified drift detection in data streams into two categories: active detection, which monitors data features over time and triggers an alarm when drift occurs, and passive detection, which ignores past data and only uses the most recent observations. The sliding window technique is a widely used passive adaptation method in the literature, such as in the HDDDM algorithm, which uses fixed-size windows to split incoming data and summarize them via histograms. The first window acts as a reference for the subsequent ones, and the Hellinger distance is used to measure the difference between the reference window and the current one for each variable. However, HDDDM requires immediate label availability, which is a limitation. To address this, Laha and Verma [16] propose OTDD (Optimal Transport-based Drift Detection), which uses Wasserstein barycenters to detect distributional changes in the supervised case. They compare the barycenters of the reference and current windows using the Kullback-Leibler divergence, signaling drift when the threshold is exceeded and updating the reference window accordingly.

The use of Wasserstein barycenters provides a robust representation of the distribution that is resistant to outliers while respecting the geometric structure and complexity of the data. The pseudo-code of the OT-based algorithm for concept drift detection (OTDD) is given in [16].

To evaluate machine learning models, precision is usually used, but in OTDD, the focus is on real-world situations where acquiring labels is costly and not instantaneous. Therefore, a new metric

is needed that accounts for both precision and the number of label acquisitions. Laha and Verma [16] proposed a new measure called the Trade-off Criterion (TC) to balance the method's accuracy and labeling requirements:

$$TC = \lambda \text{Acc} - (1 - \lambda)\text{LabelsReqd}$$

where Acc is the accuracy of the method, LabelsReqd is the fraction of total windows in a data stream for which labels are required by the method, and $\lambda \in [0, 1]$ is a constant that controls the relative importance given to each term. The ideal classifier would be one that achieves an accuracy of 1 without requiring any additional labels after learning from the initial data window in the absence of concept drift.
The evaluation of the proposed OTDD algorithm on both synthetic and real datasets demonstrates its superiority over competing algorithms, including batch, stream, and HDDDM, in terms of the trade-off between accuracy and real-time labeling requirements.

In [17], the authors introduced ADAPT, a novel approach for detecting customs fraud that balances two key concepts: exploitation (detecting known fraud) and exploration (discovering new fraudulent behaviors). This method adapts to changes in trade patterns and addresses challenges such as confirmation bias and model performance degradation due to concept drift caused by shifts in importers and goods.
ADAPT measures the dissimilarity between new and historical data distributions to detect drift. At each timestamp, customs officers use a decision-making strategy to select items for inspection based on a model trained on labeled data. The exploration ratio, influenced by model performance and detected drift, helps determine how much new data to explore.
Using OT, ADAPT calculates the minimal cost to transport current data distributions to historical ones, allowing for precise drift measurement. Experiments with real data from four African countries demonstrate that ADAPT effectively adapts to new information and identifies emerging fraud patterns, underscoring the importance of detecting concept drift in maintaining the efficacy of fraud detection systems

4 Solution

In real-world data flows, data distribution is dynamic. New concepts can emerge, and existing classes or clusters may either evolve or disappear, often leading to reduced predictive model accuracy. To address these challenges, we propose a solution that allows applications using either supervised or unsupervised machine learning models to monitor incoming data and detect concept drift by identifying both its type and the time of its occurrence.
In this section, we describe our approach to designing a solution named **Relio (Reliable Outcomes)**, which ensures the detection of distribution changes and maintains the accuracy of predictive models. We also outline the system's architecture.

4.1 Global Architecture

Relio serves as an intermediate layer between the application's inputs and the ML model in production, preventing performance degradation due to concept drift. It operates with a set of user-defined parameters including window size, alert threshold, detection threshold, OT distance, cost function, and stability threshold. As illustrated in figure 6, Relio comprises three core components: the initializer, the concept drift detector, and the concept drift type identifier.

- The initializer sets up the reference distribution (representing the latest stable concept) and the current distribution (from the sliding window).
- The concept drift detector compares these distributions using OT metrics to identify any changes.
- The concept drift type identifier classifies the type of detected drift based on its characteristics, aiding the selection of the most appropriate model adaptation strategy.

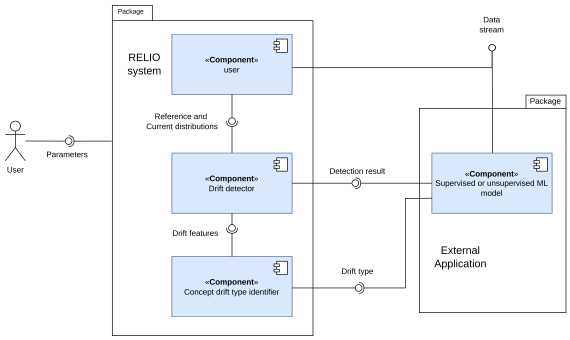

Figure 6: RELIO global architecture

4.2 Initialization Phase

The first phase of Relio involves initializing two critical entities for drift detection as shown in figure 7: the reference distribution and the current window's distribution. To detect concept drift, these distributions are compared. The sliding window approach is adopted, ensuring that only recent data is considered, with older data being discarded [15]. The window size, specified by the user, can be based on either physical or logical time.

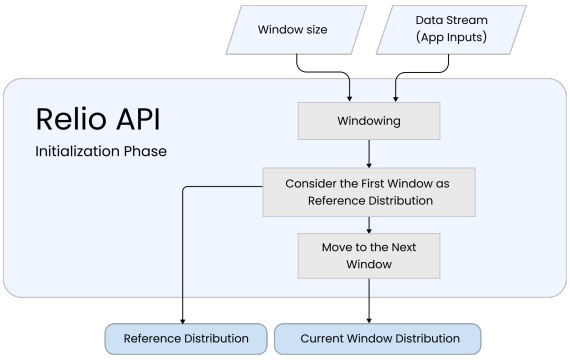

Figure 7: Initialization Phase

In this phase, the incoming data stream is segmented into windows. The first window serves as the reference distribution, while subsequent windows represent the current distribution. These two distributions are then passed to the drift detection phase.

4.3 Concept Drift Detection Phase

Concept drift is detected by comparing the current distribution with the reference using the Wasserstein distance [2], a widely recognized metric for quantifying distributional changes. This distance measures the minimum cost required to transport one distribution to another. The choice of the order p of the Wasserstein distance significantly impacts sensitivity to changes, with smaller values being more responsive to small differences, and larger values prioritizing major shifts. Additionally, the cost function $d(x, y)$ must be specified to measure the distances between points in the distributions. In our approach, we employ common cost functions : Euclidean, standardized Euclidean, and Mahalanobis distances. To enhance the detection process, we define two thresholds:

- The detection threshold identifies significant changes requiring full model retraining.
- The alert threshold detects minor changes, prompting partial model retraining to prevent future performance degradation.

Once the Wasserstein cost is calculated, it is compared to the alert and detection thresholds:

- **No change:** If the cost is below the alert threshold, the distributions are similar, and no drift is detected. The reference distribution is updated with a random element from the current distribution to ensure ongoing accuracy.
- **Minor change:** If the cost exceeds the alert threshold but remains below the detection threshold, a minor change is detected. Partial model retraining is suggested to update the ML model efficiently without full retraining.
- **Significant change:** If the cost exceeds the detection threshold, a major drift has occurred, requiring full model retraining. The reference distribution is archived, and the current distribution becomes the new reference for future comparisons.

The figure 8 illustrates the steps of the detection phase.

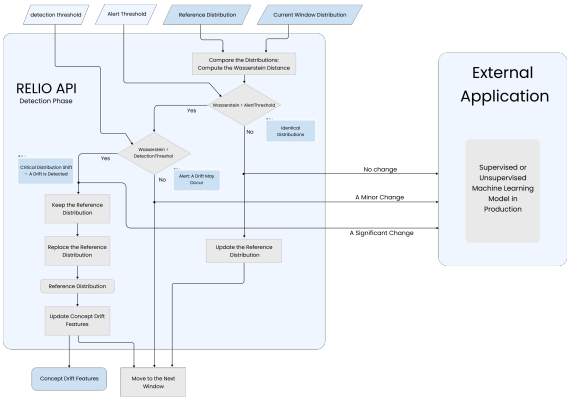

Figure 8: Concept Drift Detection Phase

4.4 Concept Drift Types Identifier

In this phase, Relio identifies the type of concept drift that has occurred, helping to choose the most appropriate strategy for updating the model. Relio distinguishes between four main types of drifts: sudden, incremental, gradual, and recurrent. Understanding the type of drift is crucial, as each has different characteristics and implications for how the model should be updated. The figure 9 shows the steps of the concept drift type identification.

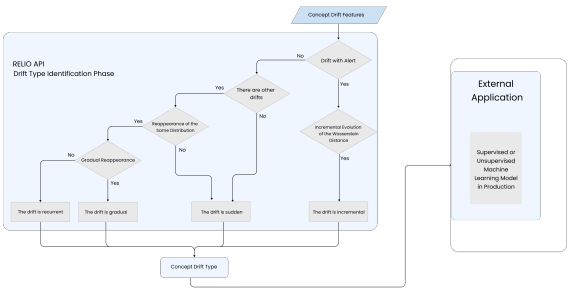

Figure 9: Concept Drift Type Identification Phase

- **Sudden drift** occurs when there is an abrupt change in the data distribution, reflected by a sharp increase in the OT distance between two consecutive windows. This type of drift happens without any prior warning, meaning that the model's performance can drop quickly if not addressed. To ensure stability, we use a parameter called the stability threshold, which defines a waiting period after detecting the drift. During this period, if no further drifts occur, it confirms that the data has fully transitioned to a new concept. This stability check helps avoid unnecessary retraining triggered by short-term fluctuations.

- **Incremental drift** is characterized by a slow, step-by-step change in the data distribution. In this case, the Wasserstein distance gradually increases over multiple windows, crossing the detection threshold at a later point. To identify an incremental drift, Relio checks if an alert was triggered in the previous window, and whether the distance between distributions is steadily increasing. This gradual change allows for partial model updates over time, reducing the need for complete retraining.

- **Gradual drift** happens when two distributions co-exist for some time, with one fading out as the other becomes dominant. In this type of drift, the data transitions smoothly from the first concept to the second. Relio monitors these changes by checking that both distributions are active simultaneously, and that the first distribution is shrinking while the second is growing. By tracking the length of time each concept is present, Relio ensures that the transition between distributions is gradual and smooth, avoiding sudden changes in model behavior.

- **Recurrent drift** refers to a situation where a previous data distribution reappears after a period of time. This type of drift is not periodic, but it indicates that the system cycles back to a previously

observed concept. To detect a recurrent drift, Relio compares the new distribution to the previously stored reference distributions, ensuring that the same pattern is returning. Since recurrent drift doesn't follow a fixed schedule, Relio also verifies that the drift is not gradual by checking that the first distribution is not disappearing as the second reappears.

5 Results

In this section, we evaluate the effectiveness of Relio in detecting and identifying concept drift. We tested our solution using synthetic, benchmark, and real-world datasets, comparing its performance against baseline methods. The results were analyzed using appropriate metrics to confirm Relio's effectiveness and pinpoint areas for potential improvement.

5.1 Datasets

Synthetic Datasets : We used the Massive Online Analysis (MOA) tool to simulate concept drift in data streams [18]. While MOA alters the relationships between inputs and outputs to create drifts, our approach focuses on detecting changes in data distribution. We simulated these changes in a controlled environment and applied Relio to evaluate its capability to detect and identify various types of drift and their effects on machine learning model performance.
To simulate concept drift, we proposed that it occurs when a new class or cluster appears, or when two classes overlap, resulting in changes to the data distribution. We used the Iris dataset, which has three classes, to simulate different types of drift. First, we separated the dataset into three dataframes, one for each class. We then created data windows (we used a sliding window [15]) by selecting varying amounts of data from each class, which allowed us to simulate four types of concept drift. For sudden drift, we introduced a new class abruptly into the later data windows. By combining these windows, we generated a new dataset containing both sudden and incremental drift. We also simulated recurrent drift by repeating the same distribution and gradual drift by gradually transitioning between two distributions.
Additionally, we utilized synthetic datasets from the literature to simulate concept drift [19]. One example is the Insects Sudden dataset, which simulates sudden drift by changing temperature. This dataset comprises 24,150 instances and 33 variables, with sudden drift introduced during the experiments.
Real Datasets : To further validate our solution, we employed two real-world datasets :

- *Ozone:* This dataset contains air quality measurements collected between 1998 and 2004 from the Houston, Galveston, and Brazoria regions [20]. The goal is to predict ozone levels eight hours in advance. It includes 2,534 instances, 72 attributes, and two class labels: ozone day and normal day.

- *Asfault:* This dataset, described by [21], tackles a 5-class problem by evaluating road quality across five categories: good, average, fair, obstacles, and poor. It consists of 8,563 instances and 64 features.

5.2 Detection Results

In this section, we test our solution using the datasets mentioned earlier. For each dataset, we first show the data flow to identify distribution changes. Next, we display the optimal transport distance between the reference distribution and the current data window. We also compare Relio with the EMD and JS methods for detecting concept drift.
To evaluate how different types of concept drift affect machine learning model performance, we monitor two models of the same type (either supervised or unsupervised). Both models are trained on the same data with the same hyperparameters, which were determined using grid search approach. The first model is non-adaptive, meaning it does not adjust to changes in the data distribution. The second model is adaptive, which means it retrains partially for

minor changes and fully for significant ones.

5.2.1 Simulated Gradual Concept Drift Dataset

After simulating gradual drift, we created a stream of 340 data points, illustrated in Figure 10. This figure shows the data distribution of the first Principal Component (PC), with 90.04% of the information displayed against the data index. We used two distributions, with the new distribution starting to increase at data index 120. The first distribution gradually decreases and is fully replaced by the second distribution starting from data index 240.

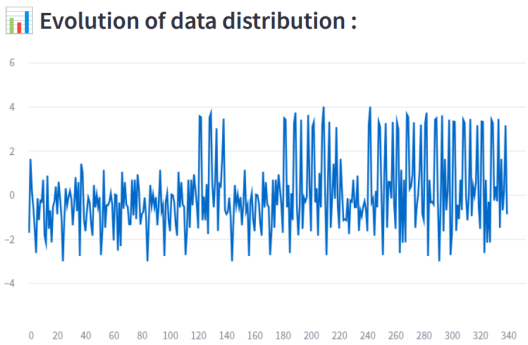

Figure 10: Iris Gradual Data Distribution

Each transition between the two distributions should result in an increase in the comparison metric. We check this by applying our solution, using the second-order Wasserstein distance with the standardized Euclidean distance as the cost function [1]. We considered windows of 20 data points and set a stability threshold of 4 windows. This threshold helps us detect when the first distribution is being replaced by the second. The alert and detection percentages were set at 20% and 40%, respectively. Figure 11 shows how the second-order Wasserstein distance evolves between the reference distribution and the current window as the window index changes. We observe increases in the comparison metric at window indices 5, 6, 8, 10, and 11, which correspond to the transitions between the two distributions.

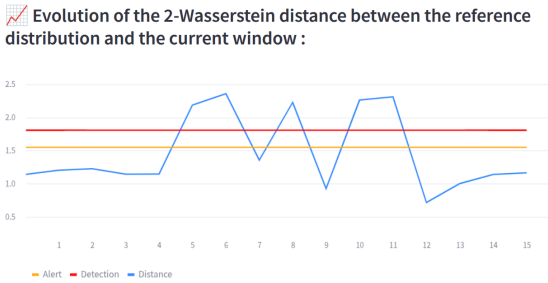

Figure 11: 2-Wasserstein Distance between the reference and the current distributions

Once the second distribution takes over, we see that the values of the comparison 2-Wasserstein distance are low.

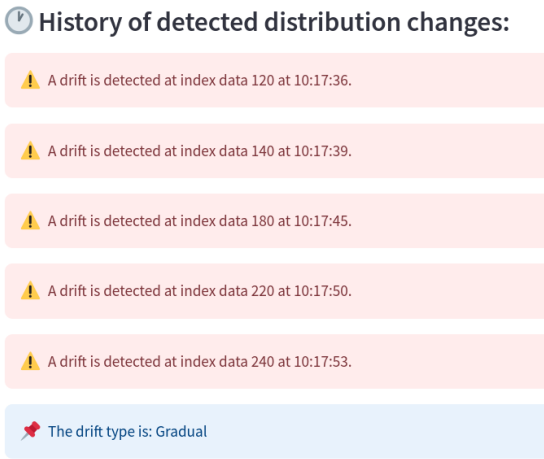

Figure 12: History of detected distribution changes

Our solution produced the results shown in Figure 12, demonstrating that all distribution changes were detected correctly, including the identification of the gradual drift type. We then compared Relio with EMD and JS on the first PC of this dataset, which has a representation

quality of 90.04%. All three methods successfully detected drift as it occurred. However, Relio excelled in identifying the type of drift, as shown in Figure 13.

Figure 13: Comparaison between Relio, EMD and JS

To study the impact of gradual drift on supervised machine learning, we tracked the performance of two Stochastic Gradient Descent (SGD) classifiers. Figure 14 illustrates their performance, with the red line representing the non-adaptive model and the blue line for the adaptive model. Initially, both models showed stable accuracy around 0.5. However, we noticed a drop in accuracy at window 5, which corresponds to the distribution change. The 6th window reflects a return to the first distribution, leading to improved performance. For the adaptive model, we observed fluctuations in performance due to the overlapping classes in the Iris dataset. In general, the non-adaptive model performed better during the first distribution but struggled with the new one. Overall, the adaptive model outperformed the non-adaptive model, highlighting that gradual drift can negatively impact model performance if not addressed.

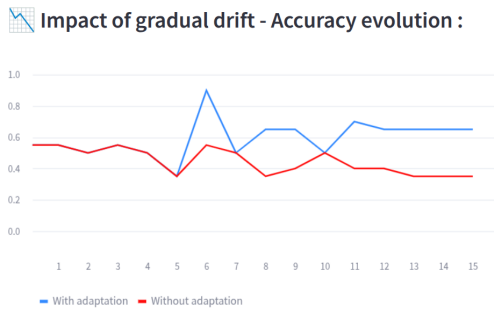

Figure 14: Accuracy Evolution of adaptive and non-adaptive models

5.2.2 Insects Sudden

We tested our solution using the Insects dataset featuring sudden drift. A visual representation of part of the data from the first PC, with a representation quality of 35.99%, is shown in Figure 15. At index 2000, we notice a sudden change in the distribution, which we verify with our solution.

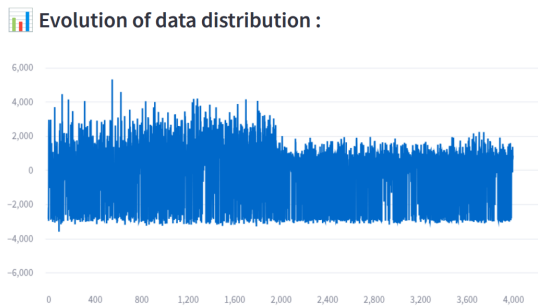

Figure 15: Insects Sudden Data Distribution

We applied our approach using the first-order Wasserstein distance with the standardized Euclidean distance as the cost function. The window size was set to 200 data items, with alert and detection thresholds of 50% and 70%, respectively. Figure 16 illustrates the evolution of the Wasserstein distance between the reference distribution and the current window. A sharp increase occurs at window index 9, which starts

with the data from index 2000, followed by a decrease in distance, indicating the similarity of subsequent distributions.

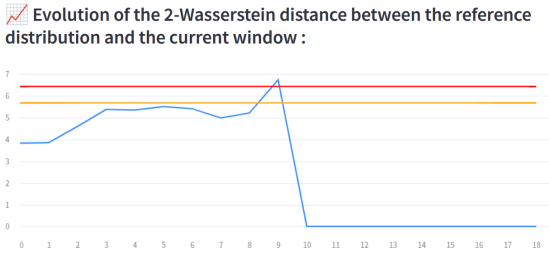

Figure 16: 2-Wasserstein Distance between the reference and the current distributions

In comparing the three methods on the first PC of this dataset, we obtained the results displayed in Figure 17. These results differ from the previous ones as we are detecting changes in the first PC, which accounts for only 36% of the information.

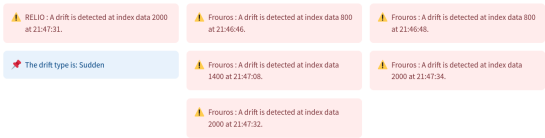

Figure 17: Comparaison between Relio, EMD and JS

The EMD and JS methods detected drift at data level 600, while our solution did not. However, both Relio and JS identified drift at data level 800, while EMD did not. Later, Relio and EMD detected drift at data 1400, whereas JS did not. Eventually, all three methods detected drift from data 2000 onward, and Relio indicated that all three drifts were sudden.

To assess the impact of sudden drift on supervised models, we compared the accuracy of two SGD classifiers: a non-adaptive model and an adaptive model. The evolution of their accuracy is shown in Figure 18. The non-adaptive model's accuracy (in red) sharply declined when sudden drift occurred and was not accounted for. In contrast, the adaptive model, which retrained upon detecting drift, performed better, as indicated by the blue line.

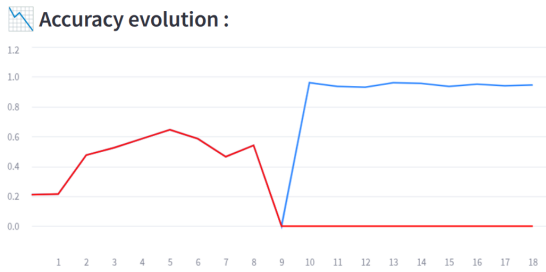

Figure 18: Accuracy Evolution of adaptive and non-adaptive models

5.2.3 Ozone

For this test, we used the second-order Wasserstein distance with the standardized Euclidean distance as the transport cost function. We set the window size to 150 data points, with an alert threshold of 50% and a detection threshold of 70%. Figure 19 illustrates the evolution of the 2-Wasserstein distance between the reference distribution and the current window.

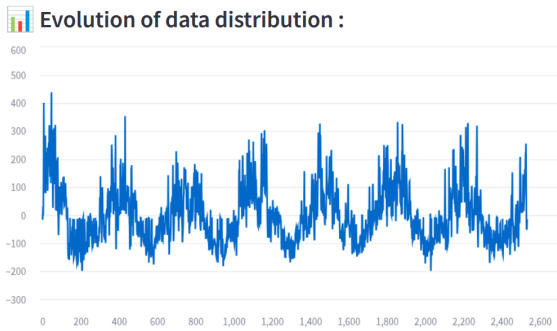

Figure 19: Ozone Data Distribution

The data visualization reveals a reappearance of the same distribution, consistent with the behavior of the comparison metric. Relio's output

indicates that the drift is recurrent, as shown in Figure 20.

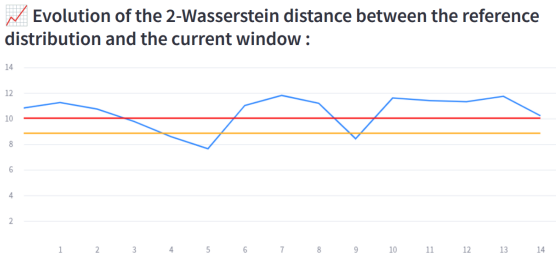

Figure 20: 2-Wasserstein Distance between Reference and Current Window

In comparing Relio, EMD, and JS, we found that all three methods detected changes at the same data points, as illustrated in Figure 21.

Figure 21: Comparaison between Relio, EMD and JS

Figure 22 displays the performance evolution of two SGD classifiers: one that is non-adaptive to changes and the other that is adaptive.

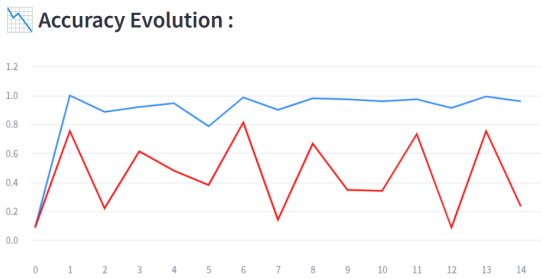

Figure 22: Accuracy Evolution of Adaptive and Non-adaptive models

We observe that the non-adaptive model's performance (in red) fluctuates due to the periodic reappearance of the same distribution, reflecting the impact of recurrent drift. This fluctuation renders the model less effective when the new distribution appears. In contrast, the adaptive model (in blue) maintains better and more stable performance.

5.2.4 Asfault

Figure 23 shows a visualization of the first PC of the Asfault dataset, which captures 27.18% of the total information. We observe several distribution changes in this visualization.

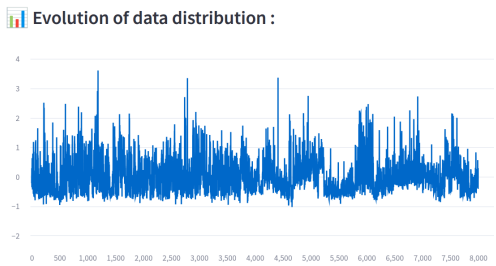

Figure 23: Asfault Data Distribution

When applying Relio for drift detection on this dataset, the results are displayed in Figure 24. It shows the evolution of the second-order Wasserstein distance using the standardized Euclidean distance as the cost function. We set alert and detection thresholds at 10% and 20%,

respectively, and used a window size of 500 data points. The optimal transport distance increased incrementally at windows 2 and 7, corresponding to data indices 1500 and 4000. Additionally, drifts were detected at data indices 5000, 5500, 6500, and 7000.

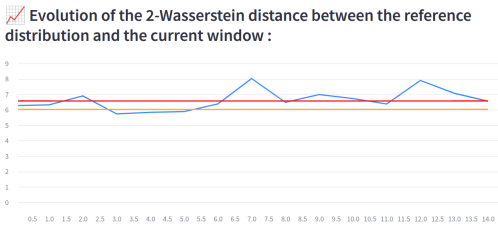

Figure 24: 2-Wasserstein Distance between Reference and Current Windows

When comparing the drift detection methods (Relio, EMD, and JS) on the first PC of this dataset, the results show that all three methods detected most of the drift events. Figure 25 illustrates part of this comparison, where all methods returned similar results except at the 3500 index, where EMD failed to detect drift. Relio also raised an alert at data index 1000, but no drift was detected afterward.

Figure 25: Comparaison between Relio, EMD and JS

To analyze the impact of drift, we compared the performance of two clustering models. The first model was trained only on the initial data, while the second model adapted each time drift was detected. Figure 26 shows the performance of both models. The red line represents the performance of the first model, which deteriorated whenever drift occurred. In contrast, the second model, which followed an adaptation strategy, improved its performance over time.

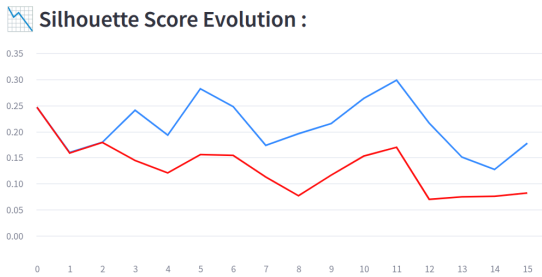

Figure 26: Silhoutte Score evolution of adaptive and non-adaptive models

Dataset	**Drift impact**	**Performances without adaptation**	**Performances with adaptation**	**Drift type**
Simulated Gradual Concept Drift Dataset	Destabilization	0.45	0.59	Gradual
Insects	Sudden destabilization	0.22	0.67	Sudden
Ozone	Notable destabilization	0.85	0.91	Recurrent
Asfault	Notable destabilization	0.13	0.21	/

Table 1: Tests recapitulation

Dataset	**Comparison (JS, EMD and RELIO)**
Simulated Gradual Concept Drift Dataset	JS, EMD et RELIO detected all the drifts, in the same positions. RELIO detected the drift type.
Insects Sudden	JS, EMD and RELIO detected the drift, but in different positions. RELIO detected the exact type of the drift (sudden).
Ozone	JS, EMD and RELIO detected all the drifts, in the same positions. RELIO detected the exact type of the drift (Reccurent).
Asfault	JS, EMD and RELIO detected the drift, but in different positions. RELIO sent alerts when small drifts occurred.

Table 2: Methods Comparison

6 Conclusion and Perspectives

In conclusion, machine learning (ML) models in streaming environments face significant challenges, particularly the issue of concept drift, which can severely impact their performance. This degradation can occur for various reasons, such as evolving user behaviors, changes in business processes, or errors in data collection. Con-

cept drift can result in models that no longer reflect reality, leading to inaccurate predictions and analyses. Hence, it is crucial to detect these changes swiftly and respond effectively.

Traditional approaches often monitor model performance to detect drift, but this can lead to delayed detection. To address this, we propose a proactive solution based on distribution comparison using OT. In this work, we introduced Relio, an API designed to detect concept drift by leveraging OT metrics known for their precision and efficiency in comparing distributions. What sets our solution apart is its ability to identify different types of drift and handle multidimensional distributions while preserving their structure.

Through extensive testing on both synthetic and real-world datasets, we demonstrated Relio's effectiveness in detecting concept drift as it occurs, accurately identifying its type, and maintaining the performance of ML models before they are affected by distributional changes.

This work also lays the foundation for future research and development. Promising directions include adapting model responses based on the type of drift, improving detection thresholds, exploring additional metrics and cost functions in OT, and interpreting the impact of drift on key features. Relio's flexibility and robustness offer a promising avenue for enhancing the adaptability and precision of concept drift detection in streaming environments.

References

[1] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. IEEE transactions on knowledge and data engineering, 31(12), 2346-2363.

[2] Monge, G. (1781). Mémoire sur la théorie des déblais et des remblais. Mem. Math. Phys. Acad. Royale Sci., 666-704

[3] Montesuma, E. F., Mboula, F. N., & Souloumiac, A. (2023). Recent advances in optimal transport for machine learning. arXiv preprint arXiv:2306.16156.

[4] Redko, I., Habrard, A., & Sebban, M. (2017). Theoretical analysis of domain adaptation with optimal transport. In Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2017, Skopje, Macedonia, September 18–22, 2017, Proceedings, Part II 10 (pp. 737-753). Springer International Publishing.

[5] Barbe, D., Borgnat, P., Gonçalves, P., & Sebban, M. (2019, August). Transport Optimal sous Contrainte de Régularité pour l'Adaptation de Domaines entre Graphes avec Attributs. In GRETSI 2019-XXVIIème Colloque francophonede traitement du signal et des images (pp. 1-4).

[6] Kerdoncuff, T., Emonet, R., & Sebban, M. (2021, January). Metric learning in optimal transport for domain adaptation. In International Joint Conference on Artificial Intelligence.

[7] Leite, D., Baptista, D., Ibrahim, A. A., Facca, E., & De Bacco, C. (2022). Community detection in networks by dynamical optimal transport formulation. Scientific Reports, 12(1), 16811.

[8] Kandasamy, K., Neiswanger, W., Schneider, J., Poczos, B., & Xing, E. P. (2018). Neural architecture search with bayesian optimisation and optimal transport. Advances in neural information processing systems, 31.

[9] Torres, L. C., Pereira, L. M., & Amini, M. H. (2021). A survey on optimal transport for machine learning: Theory and applications. arXiv preprint arXiv:2106.01963.

[10] Torres, L. C., Pereira, L. M., & Amini, M. H. (2021). A survey on optimal transport for machine learning: Theory and applications. arXiv preprint arXiv:2106.01963.

[11] Klinkenberg, R. (2004). Learning drifting concepts: Example selection vs. example weighting. Intelligent data analysis, 8(3), 281-300.

[12] Lemaire, V., Salperwyck, C., Bondu, A. (2015). A Survey on Supervised Classification on Data Streams. In : Zimányi, E., Kutsche, RD. (eds) Business Intelligence. eBISS 2014. Lecture Notes in Business Information Processing, vol 205. Springer, Cham

[13] Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boisbunon, A., Chambon, S., & Vayer, T. (2021). Pot : Python optimal transport. The Journal of Machine Learning Research, 22(1), 3571-3578.

[14] Cuturi, M. (2013). Sinkhorn distances : Lightspeed computation of optimal transport. Advances in neural information processing systems, 26.

[15] Zhao, D., Koh, Y. S. (2020). Feature drift detection in evolving data streams. In Database and Expert Systems Applications: 31st International Conference, DEXA 2020, Bratislava, Slovakia, September 14–17, 2020, Proceedings, Part II 31 (pp. 335-349). Springer International Publishing.

[16] Laha, A. K., Verma, S. (2021). Optimal Transport based Drift Detection for Sensor Streams: Method and Applications in Transportation (No. WP 2021-09-01). Indian Institute of Management Ahmedabad, Research and Publication Department.

[17] Mai, T. D., Hoang, K., Baigutanova, A., Alina, G., Kim, S. (2021, December). Customs fraud detection in the presence of concept drift. In 2021 International Conference on Data Mining Workshops (ICDMW) (pp. 370-379). IEEE.

[18] Bifet, A., Holmes, G., Pfahringer, B., Kranen, P., Kremer, H., Jansen, T., Seidl, T. (2010, September). Moa: Massive online analysis, a framework for stream classification and clustering. In Proceedings of the first workshop on applications of pattern analysis (pp. 44-50). PMLR.

[19] Souza, V. M., dos Reis, D. M., Maletzke, A. G., Batista, G. E. (2020). Challenges in benchmarking stream learning algorithms with real-world data. Data Mining and Knowledge Discovery, 34(6), 1805-1858.

[20] Zhang,Kun, Fan,Wei, and Yuan,XiaoJing. (2008). Ozone Level Detection. UCI Machine Learning Repository. https://doi.org/10.24432/C5NG6W

[21] Souza, V. M. A. (2018). Asphalt pavement classification using smartphone accelerometer and complexity invariant distance. Engineering Applications of Artificial Intelligence, 74, 198-211